Backend Notes: Node.js, Express.js, MongoDB and Testing

Oct 28, 2026 · @Mr V

How to use these notes

By the end of these notes you will have built the backend for **StudentDesk**, a React app that lists, adds and deletes students. Your friend's React app (the frontend) is already built. You will build the part behind it: a Node.js and Express.js API that saves students in MongoDB, plus tests that check it.

Every concept follows the same three parts:

1. **Explanation:** why we need this, what result we get, and what each word in the code means.
2. **Example code:** a short, simple example you type and run.
3. **Task (StudentDesk):** you use the same idea to build one piece of the StudentDesk backend.

The examples and the tasks use the same ideas and the same names, so once you understand the example, the task is the same pattern applied to the real project.

What the React app expects from your backend

The frontend has this line in `src/App.jsx`:

```
const API = "http://localhost:5000/api/students";
```

So your backend must match these exactly:

Need	Value
Server address	`http://localhost:5000`
List students	`GET /api/students`
Add a student	`POST /api/students` (send back the saved student)

Need	Value
Delete a student	`DELETE /api/students/:id`
Fields	`name`, `usn`, `email`, `department`
Departments	`CSE`, `ISE`, `ECE`, `ME`
Unique id	`_id` (MongoDB creates it)

The finished project folder

```
studentdesk/
├── frontend/                  ← the React app (do not change)
└── backend/                   ← what you build
    ├── controllers/studentController.js
    ├── models/Student.js
    ├── routes/studentRoutes.js
    ├── tests/jest/  and  tests/mocha/
    ├── app.js
    ├── index.js
    ├── .env
    ├── .gitignore
    └── package.json
```

Practice examples go in a separate `practice/` folder, so they never break the real project.

Setup 1: Install Node.js, VS Code and Git

Explanation

Why: JavaScript normally runs only inside a browser. **Node.js** lets it run on your computer, which is what a backend needs. Installing Node.js also installs **npm** (Node Package Manager), which downloads libraries such as Express for you. **VS Code** is where you write the code, **Thunder Client** lets you send test requests to your API, and **Git** saves versions of your project and uploads it to GitHub.

What you get: the commands `node`, `npm` and `git` work in your terminal.

Why LTS: LTS means Long Term Support. It is the stable version that companies use, so choose LTS rather than "Current".

Install Node.js (includes npm)

Windows

1. Go to **nodejs.org** and download the **LTS** installer (`.msi`).
2. Run it and keep the default options. Keep **"Add to PATH"** ticked.
3. Close any open terminal and open a new one, so it sees the new commands.

Mac

1. Go to **nodejs.org**, download the **LTS** installer (`.pkg`) and run it.
2. Or, if you use Homebrew: `brew install node`

Linux (Ubuntu)

The easiest reliable way is **nvm** (Node Version Manager):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh |
bash
```

Close and reopen the terminal, then:

```
nvm install --lts
```

Check it worked (all systems):

```
node -v
npm -v
```

Each command should print a version number, for example `v24.x.x` and `11.x.x`. If you see "command not found", close and reopen the terminal; on Windows, reinstall with "Add to PATH" ticked.

Install VS Code and Thunder Client

1. Download **VS Code** from **code.visualstudio.com** and install it.
2. Open VS Code → click the **Extensions** icon in the left bar → search **Thunder Client** → **Install**. A lightning-bolt icon appears in the left bar.
3. Optional but helpful: install **Error Lens**, which shows errors right on the line where they happen.

Install Git

- **Windows:** download from **git-scm.com** and install with the default options.
- **Mac:** run `git --version` in the terminal; if Git is missing, macOS offers to install it.
- **Linux (Ubuntu):** run the two commands below.

```
sudo apt update
sudo apt install git
```

Tell Git who you are (once per computer). Use the same email as your GitHub account:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Check: `git --version` prints a version.

Task (StudentDesk)

- [ ] `node -v`, `npm -v` and `git --version` all print a version
- [ ] VS Code and Thunder Client are installed
- [ ] Create a GitHub account at **github.com** if you don't have one

Setup 2: MongoDB Atlas (cloud.mongodb.com)

Explanation

Why: a backend needs somewhere to keep data permanently. If you store students in a JavaScript array, they disappear every time the server restarts. **MongoDB** is a database that stores data as JSON-like documents, so it feels natural in JavaScript. **MongoDB Atlas** is MongoDB running in the cloud, so you don't install a database on your laptop.

What you get: a free database in the cloud, a username and password to log into it, and a **connection string**: the address your Node.js code uses to reach it.

Key words:

Word	Meaning
Cluster	Your database server in the cloud

Word	Meaning
Database user	The username and password your code uses (not your Atlas login)
Network access (IP access list)	Which computers are allowed to connect
Connection string	The full address of your database, used in `.env`

Step 1: Create an account and a free cluster

1. Go to **cloud.mongodb.com** and sign up (Google sign-in is fine).
2. On **Project Overview**, click **+ Create**.
3. Choose **Free** (not M10 or Flex, which cost money).
4. Name: keep **Cluster0**. Provider: **AWS**. Region: **Mumbai (ap-south-1)** if available, otherwise any Asia region.
5. Under **Quick setup**: keep **Automate security setup** ticked, and **untick Preload sample dataset** (it fills your database with practice data you don't need).
6. Click **Create Deployment**. The cluster takes a few minutes to start.

Step 2: Save the database user

A window **"Connect to Cluster0"** appears. Atlas has already:

- added **your current IP address**, and
- created a **database user** with a generated password.

Click **Copy** next to the password and save it somewhere safe now. Atlas will not show it again. Then click **Choose a connection method**.

Step 3: Copy the connection string

1. Choose **Drivers**, then driver **Node.js**.
2. Copy the string. It looks like this:

```
mongodb+srv://myuser:<db_password>@cluster0.abc12.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
```

3. Ignore the `npm install mongodb` command Atlas shows; we install `mongoose`, which includes it.

Step 4: Allow access from anywhere (for class)

Only your current IP was added. If you change Wi-Fi (for example at college), your IP changes and the connection fails.

1. Left sidebar → **Database & Network Access** → **IP Access List**.
2. **Add IP Address** → **Allow Access from Anywhere** (`0.0.0.0/0`) → **Confirm**.

This is fine for learning. Never do it for a real company project.

Step 5: Edit the connection string

Make two changes to the string you copied:

1. Replace `<db_password>` with your real password. **Remove the `< >` too.**
2. Add the database name `studentdesk` right after `.net/` , before the `?` .

Before: `mongodb+srv://myuser:<db_password>@cluster0.abc12.mongodb.net/?retryWrites=true...`

After: `mongodb+srv://myuser:Abc123xyz@cluster0.abc12.mongodb.net/studentdesk?retryWrites=true...`

You will paste this into the `.env` file in Setup 4.

Where to see your data later

Atlas → **Data Explorer** → click the arrow next to **Cluster0** → **studentdesk** → **students**. The `studentdesk` database only appears after the first student is saved.

Task (StudentDesk)

- [ ] Free cluster **Cluster0** is created
- [ ] Database user password is saved somewhere safe
- [ ] `0.0.0.0/0` is in the IP Access List
- [ ] Your connection string is edited: real password, no `< >` , and `/studentdesk` after `.net`

Setup 3: Project folders, package.json and packages

Explanation

Why: every Node.js project needs a `package.json` file. It is the project's ID card: its name, the commands to run it, and the list of packages it needs. When someone else downloads your project, they run `npm install` and npm reads this list to install everything.

What you get: a `backend` folder with `package.json`, a `node_modules` folder holding the installed packages, and `package-lock.json` recording the exact versions.

Why each package:

Package	Why we need it
`express`	Makes routes and servers easy to write
`cors`	Lets the React app (port 5173) call our API (port 5000)
`mongoose`	Talks to MongoDB and checks our data rules
`dotenv`	Reads secret values from the `.env` file
`nodemon`	Restarts the server automatically when you save a file
`jest`, `mocha`, `chai`, `supertest`	Testing tools (used at the end)

Why `-D`: `npm install -D` saves a package under **devDependencies**: tools needed only while developing (nodemon, test tools), not to run the app.

Important: work on your computer's main drive (for example your home folder or Documents). On Linux, an external or NTFS drive can block programs from running and give `Permission denied` for `nodemon` or `vite`.

Example code

Make the folders and install everything, from a terminal:

```
mkdir studentdesk
cd studentdesk

# put the React app here as a folder named frontend
# (unzip your friend's project and rename it to frontend)

mkdir backend
cd backend
npm init -y
npm install express cors mongoose dotenv
npm install -D nodemon jest supertest mocha chai
```

Then open `backend/package.json` and replace its `"scripts"` section with:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js",
  "test": "jest --verbose",
  "test:mocha": "mocha tests/mocha"
},
```

Your `package.json` will look roughly like this (version numbers may differ):

```
{
  "name": "backend",
  "version": "1.0.0",
  "main": "index.js",
  "type": "commonjs",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest --verbose",
    "test:mocha": "mocha tests/mocha"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.9",
    "express": "^5.3.0",
    "mongoose": "^9.11.4"
  },
  "devDependencies": {
    "chai": "^6.3.0",
```

```
    "jest": "^30.5.2",
    "mocha": "^12.0.3",
    "nodemon": "^3.1.14",
    "supertest": "^7.3.1"
  }
}
```

What each word means:

Word	Meaning
`npm init -y`	Create `package.json`; `-y` accepts all default answers
`"scripts"`	Short names for commands. `npm run dev` runs `nodemon index.js`
`"start"`, `"test"`	Special names: you can type `npm start` and `npm test` without `run`
`"dependencies"`	Packages the app needs to run
`"devDependencies"`	Packages needed only while developing
`^5.3.0`	"Version 5.3.0 or any newer 5.x"
`"type": "commonjs"`	This project uses `require(...)`. The React project uses `import` instead; that's fine, they are separate projects

Rules for JSON: keys and text go in double quotes, a comma goes between items, and there is **no** comma after the last item inside `{ }`.

Task (StudentDesk)

- [ ] Folders: `studentdesk/frontend` (the React app) and `studentdesk/backend`
- [ ] All packages installed inside `backend`
- [ ] `scripts` section added and saved (Ctrl + S)
- [ ] Check with `npm run`: it lists `start`, `dev`, `test` and `test:mocha`
- [ ] Inside `frontend`, run `npm install` once to install React's packages

Setup 4: .env, .gitignore and a Git repository

Explanation

Why `.env`: your connection string contains your database password. If you type it directly in your code and push the code to GitHub, anyone can read it and use your database. A `.env` file keeps secrets **outside** your code. Your code reads them with `process.env.MONGO_URI`.

Why `.gitignore`: it lists files that Git must **never** save or upload. We ignore:

- `.env`, because it holds the password.
- `node_modules`, because it is huge (thousands of files) and anyone can recreate it with `npm install`.

Why Git and GitHub: Git saves snapshots ("commits") of your project, so you can see what changed and go back if something breaks. GitHub stores those snapshots online, so you can share the project and submit assignments.

What you get: a project on GitHub with all your code, but **without** your password and without `node_modules`.

Example code

1. Create `backend/.env` (right-click `backend` → New File → `.env`):

```
MONGO_URI=mongodb+srv://myuser:Abc123xyz@cluster0.abc12.mongodb.net/studentde
sk?retryWrites=true&w=majority&appName=Cluster0
PORT=5000
```

Rules: one value per line, no spaces around `=`, no quotes, and no `< >` around the password.

2. Create `studentdesk/.gitignore` (in the top folder, so it covers both frontend and backend):

```
node_modules
.env
dist
```

`dist` is the frontend's build folder, which can also be recreated.

3. Optional but kind: create `backend/.env.example` with fake values, so others know what to fill in:

```
MONGO_URI=mongodb+srv://USER:PASSWORD@cluster0.xxxxx.mongodb.net/studentdesk
PORT=5000
```

4. Make the Git repository. In a terminal inside the top `studentdesk` folder:

```
git init
git add .
git status
```

Read the `git status` list carefully. **`.env` and `node_modules` must NOT appear.** If they do, check the spelling in `.gitignore` and that it is in the `studentdesk` folder. Then:

```
git commit -m "Set up StudentDesk frontend and backend"
```

5. Put it on GitHub.

1. On **github.com**, click **+** (top right) → **New repository**.
2. Name it `studentdesk`. Leave **"Add a README"** and **".gitignore"** unticked (you already have them locally).
3. Click **Create repository**. GitHub shows commands; use these, with your own username:

```
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/studentdesk.git
git push -u origin main
```

When asked for a password, GitHub needs a **personal access token**, not your login password: GitHub → Settings → Developer settings → Personal access tokens → Generate new token. Or install **GitHub CLI** and run `gh auth login`.

6. Every time you finish a step later:

```
git add .
git commit -m "Describe what you added"
git push
```

What each word means:

Command	Meaning
`git init`	Start tracking this folder with Git (creates a hidden `.git` folder)
`git add .`	Choose all changed files for the next snapshot (`.` = this folder)
`git status`	Show what will be saved; use it to check `.env` is not listed
`git commit -m "..."`	Save the snapshot with a message
`git remote add origin URL`	Connect this folder to your GitHub repository
`git push`	Upload your commits to GitHub
`process.env.MONGO_URI`	How your code reads the value from `.env` (after `require("dotenv").config()`)

If you pushed `.env` by mistake: removing it later is not enough, because it stays in the history. Change your database password in Atlas (**Database & Network Access → Database Users → Edit**), update `.env`, and make sure `.gitignore` is correct.

Task (StudentDesk)

- [ ] `backend/.env` has `MONGO_URI` (your real string) and `PORT=5000`, saved
- [ ] `studentdesk/.gitignore` has `node_modules` and `.env`
- [ ] `git status` does not list `.env` or `node_modules`
- [ ] First commit pushed to a GitHub repository named `studentdesk`

Node 1: Running a file and modules

Explanation

Why we do this: until now you have run JavaScript only in a browser. A backend runs on a computer with no browser, so the first step is running a JavaScript file with Node. The second step is splitting code into several files. Real projects are never one huge file: your finished backend has a model file, a controller file and a routes file that use each other. Node calls each file a **module**.

What we get as a result: the terminal prints the output directly (no browser, no HTML). One file uses functions that live in another file.

Why this syntax:

Code	Why it is used
`node file.js`	Tells Node to run that file. The output appears in the terminal
`function name(...) { }`	Creates a reusable piece of code
`return`	Sends the result back to whoever called the function
`module.exports = { ... }`	Decides **what this file shares**. Anything not listed stays private
`require("./file")`	Loads another file and gives back what it exported
`"./"`	Means "this folder". Without it, Node looks in `node_modules` for a package, like `require("express")`
`const { a, b } = ...`	**Destructuring**: takes `a` and `b` out of the object into two variables
`` `Hello ${name}` ``	A **template literal**: backticks let you put a value inside text
`console.log(...)`	Prints to the terminal

If you forget to export something, the other file gets `TypeError: ... is not a function`. **If the path or file name is wrong** (even capital vs small letters on Linux), you get `Cannot find module`.

Example code

Create a `practice` folder (next to `backend`, not inside it), then these two files:

`practice/studentUtils.js`

```
function formatStudent(name, usn) {
  return `${name} (${usn})`;
}

function isCSE(department) {
  return department === "CSE";
}

module.exports = { formatStudent, isCSE };
```

`practice/app.js`

```
const { formatStudent, isCSE } = require("./studentUtils");

console.log(formatStudent("Asha", "1RV23CS001"));
console.log(isCSE("CSE"));
console.log(isCSE("ECE"));
```

Run it (inside `practice`):

```
node app.js
```

Result:

```
Asha (1RV23CS001)
true
false
```

Break it on purpose: change the last line of `studentUtils.js` to `module.exports = { formatStudent };` and run again. You get `TypeError: isCSE is not a function`, which proves `app.js` can only use what was exported. Put it back.

Task (StudentDesk)

The StudentDesk form allows only four departments. Keep that list in one module so every file can use the same list.

- [ ] Create `backend/departments.js` that exports `DEPARTMENTS = ["CSE", "ISE", "ECE", "ME"]` and a function `isValidDepartment(dep)` that returns `true` if `dep` is in the list (hint: `DEPARTMENTS.includes(dep)`)

- [ ] Create `backend/check.js` that requires it and prints `isValidDepartment("ISE")` and `isValidDepartment("MBA")`
- [ ] Run `node check.js`. Expected: `true` then `false`
- [ ] Delete `check.js` afterwards. You can use `DEPARTMENTS` later in the Student model (`enum: DEPARTMENTS`)

Node 2: Sync vs async (non-blocking)

Explanation

Why we do this: some jobs are slow: reading a file, or asking MongoDB for students. If the server **waited** for each slow job before doing anything else, 50 users would be served one by one. Node is **non-blocking**: it starts the slow job, carries on with other work, and comes back when the result is ready. Every database call in your backend works this way, so you need to understand it before writing `await Student.find()`.

An everyday picture: making Maggi. Sync = you stand at the stove staring for 2 minutes. Async = you set the table while the noodles cook, and serve them when the timer rings.

What we get as a result: the output does **not** appear in the order the code is written. The async result arrives later.

Why this syntax:

Code	Why it is used
`require("fs")`	`fs` (file system) is **built into** Node, so there's no `npm install`
`require("path")`	Builds file paths that work on Windows, Mac and Linux
`__dirname`	The folder this file is in
`fs.readFileSync(...)`	**Sync**: Node waits here until the file is read
`fs.readFile(file, "utf-8", callback)`	**Async**: Node starts reading, moves on, and runs the callback when done
`(err, data) => { }`	The **callback**: what to do when the file is ready. `err` is set if reading failed

Code	Why it is used
`"utf-8"`	Read the file as normal text (otherwise you get raw bytes)
`JSON.parse(text)`	Turns JSON text into a real JavaScript array/object
`async` / `await`	The modern way to wait for slow work. `await` pauses only this function, not the whole server

Example code

`practice/students.json`

```
[
  { "name": "Asha", "department": "CSE" },
  { "name": "Rahul", "department": "ECE" }
]
```

`practice/readStudents.js`

```
const fs = require("fs");
const path = require("path");

const file = path.join(__dirname, "students.json");

fs.readFile(file, "utf-8", (err, data) => {
  const students = JSON.parse(data);
  console.log("Students loaded:", students.length);
});

console.log("This prints first!");
```

Run it:

```
node readStudents.js
```

Result:

```
This prints first!
Students loaded: 2
```

`readFile` is written first, but its result prints second. Node didn't wait; it printed the next line while the file was being read.

The same thing with `async` / `await` (this is the style your backend uses):

```
const fs = require("fs");
const path = require("path");

async function loadStudents() {
  const file = path.join(__dirname, "students.json");
  const data = await fs.promises.readFile(file, "utf-8");
  const students = JSON.parse(data);
  console.log("Students loaded:", students.length);
}

loadStudents();
```

How this connects to StudentDesk:

This example	Your backend later
`await fs.promises.readFile(...)`	`await Student.find()`
Waits for the file	Waits for MongoDB
Other code keeps running	Other users' requests keep being served

If you forget `await`, you get a `Promise { <pending> }` instead of the data. In the backend this shows up as an empty `{}` response.

Task (StudentDesk)

- [ ] Add a third student to `practice/students.json` with your own name and department `ISE`
- [ ] Write `practice/countByDept.js` with `async` / `await` that reads the file and prints how many students are in `CSE` (hint: `students.filter(s => s.department === "CSE").length`)
- [ ] Remove the `await` and run it again. You get an error, because data is now a Promise, not text. Put `await` back

Node 3: A server with the http module

Explanation

Why we do this: a backend is a **server**: a program that keeps running and waits for **requests**, then sends back **responses**. Before using Express, we build one with only Node, so you can see what a server really does and why Express is useful.

The key ideas:

- **Client:** whoever sends a request: the browser, Thunder Client, or the React app.
- **Server:** the program that answers. `createServer` makes a server **program**, which runs inside Node on your laptop.
- **Port:** a door number on your computer. Only one program can use a port at a time. Our API will use **5000**; React uses **5173**.
- **`localhost`:** "this same computer".

What we get as a result: the terminal shows `Server running...` and **does not return to the prompt**, because the server stays on, waiting. Visiting URLs in the browser shows different answers.

The flow of one visit: you open `localhost:3000/students` → the browser sends a request → Node runs your function with `req` and `res` → your code checks `req.url` → `res.end(...)` sends the answer → the browser shows it. This repeats for **every** visit.

Why this syntax:

Code	Why it is used
`require("http")`	Node's built-in module for servers
`http.createServer((req, res) => { })`	Makes a server. **Node calls your function once for every request**
`req` (request)	What the client asked for. You **read** from it
`req.url`	The path asked for, like `"/students"`
`req.method`	The type of request: `"GET"`, `"POST"`...
`res` (response)	How you answer. You **write** into it
`res.statusCode = 404`	The status code: 200 OK, 404 Not found...

Code	Why it is used
`res.setHeader("Content-Type", "application/json")`	Tells the client the answer is JSON
`JSON.stringify(data)`	Turns a JavaScript array/object into JSON text to send
`res.end(...)`	Sends the answer and finishes. **Forget it and the browser loads forever**
`server.listen(3000, ...)`	Starts waiting on port 3000
Ctrl + C	Stops the server

Example code

`practice/server.js`

```
const http = require("http");

const students = [
  { name: "Asha", department: "CSE" },
  { name: "Rahul", department: "ECE" },
];

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);

  if (req.url === "/") {
    res.end("Welcome to StudentDesk");
  } else if (req.url === "/students") {
    res.setHeader("Content-Type", "application/json");
    res.end(JSON.stringify(students));
  } else {
    res.statusCode = 404;
    res.end("Not found");
  }
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Run `node server.js`, then open these in the browser:

URL	Browser shows	Terminal prints
`localhost:3000/`	Welcome to StudentDesk	`GET /`
`localhost:3000/students`	The JSON list of 2 students	`GET /students`
`localhost:3000/xyz`	Not found (status 404)	`GET /xyz`

You may also see `GET /favicon.ico`: the browser asks every site for its tab icon automatically.

The problem this shows: every page needs another `else if`, and you would also have to check `req.method` for GET, POST and DELETE by hand. With 20 routes this becomes a mess. **Express exists to fix exactly this.**

If you see `EADDRINUSE`, another program (usually an old server) is already on that port. Press Ctrl + C in the old terminal, or use another port.

Task (StudentDesk)

The React app asks for `/api/students` on port **5000**. Build that answer with plain Node first.

- [ ] Copy `server.js` to `practice/node-api.js`, change the port to `5000` and the path to `/api/students`
- [ ] Give each student all four React fields: `name`, `usn`, `email`, `department`
- [ ] Open `localhost:5000/api/students` in the browser and check the JSON
- [ ] Send **POST** to the same URL in Thunder Client. It still returns the list, because we never checked `req.method`. Express handles that for you
- [ ] Stop it with Ctrl + C, so port 5000 is free for the real backend

Express 1: First server and routes

Explanation

Why we do this: the plain Node server needed an `if/else` for every page and a manual check of the method. **Express** is a small library on top of Node that does this work for you. It is still a Node server underneath, with the same `req` and `res`, just with less code.

What is a route: a rule that says "when **this kind of request** arrives at **this path**, run **this function**". It has three parts:

Part	Example	Meaning
Method	`get`	What kind of request: GET read, POST add, PUT update, DELETE remove
Path	`"/api/students"`	Which URL
Handler	`(req, res) => { ... }`	The function that answers

Read `app.get("/api/students", handler)` as: "When a GET request comes for /api/students, run this function."

What we get as a result: the same pages as the Node server, with less code, plus an automatic **404** (`Cannot GET /xyz`) for any URL you didn't define, and an automatic refusal for the wrong method (`Cannot POST /about`).

Why this syntax:

Code	Why it is used
`require("express")`	Loads Express from `node_modules` (installed in Setup 3)
`express()`	Creates the **app**: the object that holds all routes and settings
`app.get(path, handler)`	Adds a GET route
`res.send("text")`	Sends text or HTML
`res.json(data)`	Sends an array/object as JSON. It does `setHeader` and `JSON.stringify` for you
`app.listen(PORT, fn)`	Starts the server; `fn` runs when it's ready
`const PORT = 5000`	Port 5000, because the React app calls `localhost:5000`

Compared with plain Node:

Need	Node only	Express
A route	`if (req.url === "/about")`	`app.get("/about", ...)`
Only GET	Check `req.method` yourself	`app.get` is GET only
Send JSON	`setHeader` + `JSON.stringify` + `res.end`	`res.json(...)`
404	Write your own `else`	Automatic

Example code

`practice/express-hello.js` (run `npm init -y` and `npm install express` inside `practice` first)

```
const express = require("express");
const app = express();
const PORT = 4000;

app.get("/", (req, res) => {
  res.send("Welcome to StudentDesk");
});

app.get("/students", (req, res) => {
  res.json([
    { name: "Asha", department: "CSE" },
    { name: "Rahul", department: "ECE" },
  ]);
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run `node express-hello.js` and visit:

URL	Result
`localhost:4000/`	Welcome to StudentDesk
`localhost:4000/students`	The JSON list

URL	Result
`localhost:4000/xyz`	`Cannot GET /xyz` (404, automatic)

Task (StudentDesk)

Start the real backend.

- [ ] Create `backend/index.js` with Express on **port 5000**
- [ ] `GET /` returns the text `StudentDesk API is running`
- [ ] `GET /api/students` returns a JSON array of 2 students, each with `name`, `usn`, `email`, `department`
- [ ] Run it with `npm run dev` (inside `backend`). Change the welcome text and save: nodemon restarts the server by itself
- [ ] Open `localhost:5000/api/students` in the browser and in Thunder Client: status **200**

Express 2: Params, query, body and CRUD

Explanation

Why we do this: almost every app does four things with data, called **CRUD**: Create, Read, Update, Delete. StudentDesk lists students (Read), adds them (Create) and deletes them (Delete). Each action is one HTTP method:

CRUD	HTTP method	StudentDesk route
Create	POST	`POST /api/students`
Read	GET	`GET /api/students` and `GET /api/students/:id`
Update	PUT	`PUT /api/students/:id`
Delete	DELETE	`DELETE /api/students/:id`

Three ways the client sends data:

Where	Example	Read it with	Used for
Route param	`/api/students/2`	`req.params.id` → `"2"`	Picking one item
Query string	`/api/students?department=CSE`	`req.query.department` → `"CSE"`	Filtering or searching
Body	`{ "name": "Meera" }`	`req.body`	New or changed data (POST, PUT)

Status codes tell the client what happened: **200** OK, **201** Created, **400** bad data, **404** not found, **500** server error.

What we get as a result: a working API you can test with Thunder Client. But the data lives in an **array in memory**, so it disappears when the server restarts. That is the problem MongoDB solves later.

Why this syntax:

Code	Why it is used
`app.use(express.json())`	Reads a JSON body into `req.body`. **Without it, `req.body` is `undefined`**
`"/students/:id"`	`:id` is a placeholder: it matches any value in that part of the URL
`req.params.id`	The value that filled `:id`. It is always **text**
`.find(s => ...)`	Gets the first item that matches
`.filter(s => ...)`	Gets a new array of all matching items (used to delete and to filter)
`.push(item)`	Adds to the end of the array
`{ _id: ..., ...req.body }`	**Spread**: copies every field from the body into the new object
`String(Date.now())`	A quick unique id from the current time
`res.status(201).json(...)`	Sets the status code, then sends JSON

Code	Why it is used
`return res...`	`return` stops the function, so it can never send two responses
`Object.assign(student, req.body)`	Copies the body's fields onto the student (the update)

We call the id `_id` because that is the name the React app reads (`student._id`), and the name MongoDB uses later.

Example code

`practice/crud.js`

```
const express = require("express");
const app = express();

app.use(express.json());

let students = [
  { _id: "1", name: "Asha", department: "CSE" },
  { _id: "2", name: "Rahul", department: "ECE" },
];

// READ all, with an optional filter: /students?department=CSE
app.get("/students", (req, res) => {
  const { department } = req.query;
  if (department) {
    return res.json(students.filter(s => s.department === department));
  }
  res.json(students);
});

// READ one: /students/2
app.get("/students/:id", (req, res) => {
  const student = students.find(s => s._id === req.params.id);
  if (!student) return res.status(404).json({ error: "Student not found" });
  res.json(student);
});

// CREATE
app.post("/students", (req, res) => {
```

```
  const newStudent = { _id: String(Date.now()), ...req.body };
  students.push(newStudent);
  res.status(201).json(newStudent);
});

app.listen(4000, () => console.log("CRUD demo on port 4000"));
```

Test it in Thunder Client:

Request	Body (JSON)	Result
GET `localhost:4000/students`	none	200, Asha and Rahul
GET `localhost:4000/students?department=CSE`	none	200, only Asha
GET `localhost:4000/students/2`	none	200, Rahul
GET `localhost:4000/students/9`	none	404, Student not found
POST `localhost:4000/students`	`{ "name": "Meera", "department": "ISE" }`	201, Meera with a new `_id`

Where things go in Thunder Client: the method in the dropdown, the id in the **URL**, and the JSON in **Body → JSON**. In JSON, every field needs a comma after it except the last one.

Break it on purpose: comment out `app.use(express.json())` and POST again. Meera comes back with only an `_id`, because `req.body` was empty. Then restart the server: Meera is gone, because the array lives in memory.

Task (StudentDesk)

Upgrade `backend/index.js` to full CRUD with an array.

- [ ] Add `app.use(express.json())` above the routes
- [ ] Make an array of 2 students with `_id`, `name`, `usn`, `email`, `department`
- [ ] Add `GET /api/students` (with `?department=` filter), `GET /api/students/:id`, `POST /api/students` (201)

- [ ] Add `PUT /api/students/:id` (404 if not found; hint: `Object.assign`) and `DELETE /api/students/:id` (hint: `filter` with `!==`, then `res.json({ message: "Student deleted" })`)
- [ ] Test all five in Thunder Client
- [ ] Commit to Git: `git add .` then `git commit -m "CRUD with array"`

Express 3: Middleware and CORS

Explanation

Why we do this: some jobs must happen for **many** requests, not just one route: printing a log line, checking who is allowed in, reading JSON, giving the React app permission. Instead of repeating that code in every route, we write it once as **middleware**: a function that runs **between** the request arriving and the route handler.

An everyday picture: middleware is the **security guard at the gate**. Every visitor passes the guard. The guard can write down their name (logger), check their ID card (`checkKey`), or turn them away, before they reach the room (the route).

The flow:

```
Request → logger (everyone) → checkKey (only /secret) → route handler → Response
```

The one rule: every middleware must either call **`next()`** (let the request continue) or **send a response** (stop it here). If it does neither, the request is stuck and the browser loads forever.

What CORS is: the React app runs on `localhost:5173` and the API on `localhost:5000`. A different port is a different **origin**. For safety, **browsers** block a page from reading another origin's data, unless that server says it's allowed. `app.use(cors())` is middleware that adds the permission header `Access-Control-Allow-Origin: *` to every response. Thunder Client is not a browser, so it never shows CORS errors. That is why an API can work in Thunder Client and still fail in React.

What we get as a result: a log line in the terminal for every request, a protected route, and a React app that is allowed to call our API.

Why this syntax:

Code	Why it is used
`app.use(fn)`	Runs `fn` for **every** request, in the order the `app.use` lines are written
`(req, res, next) => { }`	A middleware receives a third thing: `next`
`next()`	"I'm done, pass the request to the next step"
`app.get("/secret", checkKey, handler)`	Runs `checkKey` first, then `handler`, only for this route
`return res.status(401).json(...)`	Stops the request: 401 means "not allowed"
`new Date().toLocaleTimeString()`	The current time as text, for the log
`require("cors")` / `app.use(cors())`	The CORS middleware from the `cors` package
`app.use(express.json())`	Also middleware: it reads the body before your routes run

Order matters: `cors()` and `express.json()` must be **above** the routes, so they run first.

Example code

`practice/middleware.js`

```
const express = require("express");
const app = express();

// runs for EVERY request
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} at ${new
Date().toLocaleTimeString()}`);
  next();
});

// runs only on the routes it is attached to
function checkKey(req, res, next) {
  if (req.query.key !== "123") {
    return res.status(401).json({ error: "No access" });
  }
  next();
}
```

```
app.get("/", (req, res) => {
  res.send("Open to everyone");
});

app.get("/secret", checkKey, (req, res) => {
  res.send("Welcome, admin!");
});

app.listen(4000, () => console.log("Middleware demo on port 4000"));
```

Result:

Visit	Browser shows	Terminal prints	Who answered
`localhost:4000/`	Open to everyone (200)	`GET / at 10:42:05`	Route handler
`localhost:4000/secret`	`{"error":"No access"}` (401)	`GET /secret at ...`	`checkKey` stopped it
`localhost:4000/secret?key=123`	Welcome, admin! (200)	`GET /secret?key=123 at ...`	Route handler

The terminal logs **every** visit, even the rejected one, because the logger runs first.

Break it on purpose: delete `next();` from the logger. Every page now loads forever. Put it back.

Task (StudentDesk)

Connect the React app to your array-based backend for the first time.

- [ ] In `backend/index.js`, add `const cors = require("cors");` and `app.use(cors());` **above** `express.json()` and the routes
- [ ] Add a logger middleware that prints the method, URL and time
- [ ] Start the backend (`npm run dev` in `backend`) and, in a **second terminal**, the frontend (`npm run dev` in `frontend`). Open `localhost:5173`: your array students appear in the table
- [ ] Add and delete a student with the React form and watch the logger print each request

- [ ] CORS experiment: put `//` before `app.use(cors());`, save, and add a student in React. You get "NetworkError", and F12 → Console shows a CORS error. The same request works in Thunder Client. Remove the `//` again

Express 4: Router and folder structure

Explanation

Why we do this: after Express 2 and 3, `index.js` holds the data, every route, and every handler in one long file. In a real project with students, courses and teachers, that file would have hundreds of lines. We split it so **each folder has one job**:

Folder	Job	Question it answers
`routes/`	Maps a method + path to a function	*Which* function runs for this URL?
`controllers/`	The logic inside each route	*What* happens?
`models/`	Describes the data (added in the MongoDB part)	*What does* a student look like?

What we get as a result: the app works exactly as before, but the code is organised. Adding a new feature means adding a function in the controller and one line in the routes.

How a request travels through the files (GET /api/students):

1. `index.js`: `app.use("/api/students", studentRoutes)`: the URL starts with `/api/students`, so it goes to the router.
2. `routes/studentRoutes.js`: `router.get("/", getStudents)`: the rest of the path is `/` and the method is GET, so call `getStudents`.
3. `controllers/studentController.js`: `getStudents` runs and sends the response.

Why this syntax:

Code	Why it is used
`express.Router()`	A mini-app that holds one group of related routes

Code	Why it is used
`router.get("/", fn)`	Inside the router, `"/"` means the path it is mounted on (`/api/students`)
`router.delete("/:id", fn)`	Means `DELETE /api/students/:id`
`app.use("/api/students", router)`	**Mounts** the router: every URL starting with `/api/students` goes to it
`exports.getStudents = ...`	Shares a function by name from the controller file
`const { getStudents } = require(...)`	Takes that named function out in the routes file
`"../controllers/..."`	`..` means "go up one folder": from `routes/` up to `backend/` , then into `controllers/`

Tip: in VS Code, **Ctrl + click** on a function name (like `getStudents` in the routes file) jumps to the file where it's defined.

Example code

`practice/routes/greetRoutes.js`

```
const express = require("express");
const router = express.Router();

router.get("/", (req, res) => {
  res.json({ message: "Hello, students" });
});

router.get("/:name", (req, res) => {
  res.json({ message: `Hello, ${req.params.name}` });
});

module.exports = router;
```

`practice/router-app.js`

```
const express = require("express");
const greetRoutes = require("./routes/greetRoutes");

const app = express();
app.use("/api/greet", greetRoutes);

app.listen(4000, () => console.log("Router demo on port 4000"));
```

Result:

Visit	Result
`localhost:4000/api/greet`	`{"message":"Hello, students"}`
`localhost:4000/api/greet/Asha`	`{"message":"Hello, Asha"}`

The router file never writes `/api/greet`. It gets that from `app.use`.

Task (StudentDesk)

Move your array-based student routes into folders.

- [ ] Create `backend/controllers/studentController.js`: move the `students` array and the handlers there as `exports.getStudents`, `exports.addStudent`, `exports.updateStudent`, `exports.deleteStudent`
- [ ] Create `backend/routes/studentRoutes.js` with `express.Router()`: `router.get("/", getStudents)`, `router.post("/", addStudent)`, `router.put("/:id", updateStudent)`, `router.delete("/:id", deleteStudent)`
- [ ] In `index.js`, replace the student routes with `app.use("/api/students", require("./routes/studentRoutes"));`
- [ ] Check that Thunder Client and the React app still work exactly as before
- [ ] Commit to Git

MongoDB 1: Mongoose schema and model

Explanation

Why we do this: our array forgets everything when the server restarts. MongoDB keeps the data in the cloud permanently. MongoDB itself accepts any shape of data, so a typo like `nmae` or a department like `MBA` would be saved silently. **Mongoose** adds rules: a **schema** describes what a student must look like, and refuses data that breaks the rules.

MongoDB words, compared with SQL:

SQL (MySQL)	MongoDB	In StudentDesk
Database	Database	`studentdesk`
Table	Collection	`students`
Row	Document	one student
Column	Field	`name`, `usn` …
Primary key	`_id`	created automatically

A document looks just like a JavaScript object:

```
{ "_id": "6720a1f3...", "name": "Asha", "usn": "1RV23CS001", "email":
"asha@mail.com", "department": "CSE" }
```

Schema vs model: the **schema** is the blueprint (the rules). The **model** is the tool you use to work with the collection: `Student.find()`, `Student.create()` and so on.

What we get as a result: a `Student` model that the controller can use, and a guarantee that every saved student has the four fields React needs, with an allowed department.

Why this syntax:

Code	Why it is used
`require("mongoose")`	Loads Mongoose, which talks to MongoDB for us
`new mongoose.Schema({ ... })`	Describes the fields and their rules
`type: String` / `Number`	The field must be text / a number

Code	Why it is used
`required: true`	Saving fails if this field is missing
`unique: true`	No two documents can have the same value (no duplicate USN)
`enum: [...]`	Only these values are allowed
`default: "CSE"`	Used when no value is sent
`min` / `max`	Lowest and highest allowed number
`{ timestamps: true }`	Adds `createdAt` and `updatedAt` automatically
`mongoose.model("Student", schema)`	Creates the model. MongoDB stores it in the `students` collection (lower-case, plural)
`doc.validate()`	Checks the rules **without saving** (useful for testing)

Example code

This example checks rules without connecting to a database. Run `npm install mongoose` inside `practice` first.

`practice/subjectModel.js`

```
const mongoose = require("mongoose");

const subjectSchema = new mongoose.Schema({
  name: { type: String, required: true },
  credits: { type: Number, min: 1, max: 5 },
  department: { type: String, enum: ["CSE", "ISE", "ECE", "ME"], default:
"CSE" },
});

const Subject = mongoose.model("Subject", subjectSchema);

async function check(data) {
  try {
    await new Subject(data).validate();
    console.log("Valid:", data);
  } catch (err) {
    console.log("Invalid:", Object.keys(err.errors));
```

```
  }
}

check({ name: "Data Structures", credits: 4 });
check({ credits: 4 });
check({ name: "Robotics", credits: 9, department: "MBA" });
```

Run `node subjectModel.js`.

Result (the order of lines may vary, because the checks are async):

```
Valid: { name: 'Data Structures', credits: 4 }
Invalid: [ 'name' ]
Invalid: [ 'credits', 'department' ]
```

The second one has no `name` (required). The third has credits above 5 (`max`) and a department not in the `enum` list.

Task (StudentDesk)

Create the real model the React app needs.

- [ ] Create `backend/models/Student.js` (the file name starts with a **capital S**)
- [ ] Fields: `name` (String, required), `usn` (String, required, unique), `email` (String, required), `department` (String, enum `CSE`, `ISE`, `ECE`, `ME`, default `CSE`)
- [ ] Add `{ timestamps: true }` and export it with `module.exports = mongoose.model("Student", studentSchema);`

The finished file:

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    usn: { type: String, required: true, unique: true },
    email: { type: String, required: true },
    department: {
      type: String,
      enum: ["CSE", "ISE", "ECE", "ME"],
      default: "CSE",
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Student", studentSchema);
```

The four fields match the four inputs in the React form (`StudentForm.jsx`), and the `enum` matches its department dropdown.

MongoDB 2: Controller, app.js, index.js and connecting React

Explanation

Why we do this: we now replace the array in the controller with real database calls, and connect to Atlas when the server starts. We also split the server into two files:

- **`app.js`** builds the Express app (middleware + routes) and exports it, **without** starting it.
- **`index.js`** loads `.env` , connects to MongoDB, and starts the server.

The split matters for testing later: tests can load `app.js` without connecting to the database or using port 5000.

Array code vs Mongoose code: the shape of each handler stays the same; only the data line changes.

Action	Array (Express 2)	Mongoose
List	`students`	`await Student.find()`

Action	Array (Express 2)	Mongoose
Add	`students.push(...)`	`await Student.create(req.body)`
Update	`Object.assign(...)`	`await Student.findByIdAndUpdate(id, data, { new: true, runValidators: true })`
Delete	`students.filter(...)`	`await Student.findByIdAndDelete(id)`

What we get as a result: students saved in Atlas, still there after a restart, and visible in **Data Explorer → studentdesk → students**. The React app now adds and deletes real records.

Why this syntax:

Code	Why it is used
`async (req, res) => { }`	The handler waits for the database, so it must be `async`
`await`	Pause until MongoDB answers. **Forget it and you send `{}`**
`try { } catch (err) { }`	If the database call fails (bad data, bad id), send an error instead of crashing
`res.status(400).json({ error: err.message })`	400 = the client sent bad data; `err.message` tells React why
`if (!student) return res.status(404)...`	`null` means no student has that id
`{ new: true }`	Return the updated student, not the old version
`runValidators: true`	Check the schema rules (like `enum`) on updates too
`require("dotenv").config()`	Loads `.env` into `process.env`. Keep it the **first line** of `index.js`
`process.env.PORT || 5000`	Use `PORT` from `.env`, or 5000 if it's missing
`mongoose.connect(uri)`	Opens the connection to Atlas. It returns a promise

Code	Why it is used
`.then(() => { ... })`	Runs after connecting succeeds. We start the server **only then**
`.catch((err) => { ... })`	Runs if connecting fails (wrong password, IP not allowed)
`module.exports = app`	Lets `index.js` and the tests use the same app

Example code

The handler pattern, with the database call in the middle:

```
const Student = require("../models/Student");

exports.getStudents = async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

Every other handler follows the same shape: `try` → `await` one Mongoose call → send the result with the right status → `catch` sends an error.

Task (StudentDesk)

Connect StudentDesk to MongoDB Atlas.

- [ ] Replace `backend/controllers/studentController.js` with the Mongoose version below
- [ ] Create `backend/app.js` and replace `backend/index.js` as below
- [ ] Run `npm run dev`. Expected: `MongoDB connected` then `Server running on port 5000`
- [ ] Thunder Client: POST a student → **201** with an `_id`. Copy the `_id`, then DELETE `/api/students/<_id>` → 200
- [ ] Start the frontend, add students in React, restart the backend: they are still there

- [ ] Check Atlas → Data Explorer → `studentdesk` → `students`
- [ ] Commit to Git and push

`backend/controllers/studentController.js`

```
const Student = require("../models/Student");

exports.getStudents = async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

exports.addStudent = async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

exports.updateStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true });
    if (!student) return res.status(404).json({ error: "Student not found"
});
    res.json(student);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

exports.deleteStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ error: "Student not found"
});
    res.json({ message: "Student deleted" });
```

```
  } catch (err) {
    res.status(400).json({ error: "Invalid id" });
  }
};
```

`backend/routes/studentRoutes.js`

```
const express = require("express");
const {
  getStudents,
  addStudent,
  updateStudent,
  deleteStudent,
} = require("../controllers/studentController");

const router = express.Router();

router.get("/", getStudents);
router.post("/", addStudent);
router.put("/:id", updateStudent);
router.delete("/:id", deleteStudent);

module.exports = router;
```

`backend/app.js`

```
const express = require("express");
const cors = require("cors");
const studentRoutes = require("./routes/studentRoutes");

const app = express();

app.use(cors());
app.use(express.json());
app.use("/api/students", studentRoutes);

module.exports = app;
```

`backend/index.js`

```
require("dotenv").config();
const mongoose = require("mongoose");
const app = require("./app");

const PORT = process.env.PORT || 5000;

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((err) => {
    console.log("DB error:", err.message);
  });
```

How each React action reaches your code:

React (`App.jsx`)	Request	Your function
Page loads: `fetch(API)`	`GET /api/students`	`getStudents`
Save Student button	`POST /api/students`	`addStudent`
Delete button	`DELETE /api/students/:id`	`deleteStudent`

For DELETE, the id goes in the URL, not the body. And never send `_id` in a POST body: MongoDB creates it.

Run both apps in two terminals: terminal 1 `cd backend` → `npm run dev`; terminal 2 `cd frontend` → `npm run dev`. Always check which folder the prompt shows before running an npm command.

Testing 1: Why test, and Jest

Explanation

Why we do this: so far you checked your API by hand in Thunder Client: send a request, look at the result, decide if it's right. You'd have to repeat that after every change. **Tests are code that checks your code.** One command checks every route in under a second, and turns red the moment something breaks.

An everyday picture: an exam.

Exam	Testing
The questions	Your tests ("does GET return 200?")
The invigilator who runs the exam and announces pass/fail	The **test runner**
The answer key	The **assertion library** (the checker)

The tools:

Tool	Role
Jest	Runner **and** checker in one package
Mocha	Runner only (next section)
Chai	Checker only, used with Mocha (next section)
Supertest	Sends requests to your app from inside a test, like Thunder Client but automatic. Works with both

Every API test has 3 steps:

1. **Fake** the database, so tests are fast, need no internet, and never change your real Atlas data. This is called **mocking**.
2. **Send** a request with Supertest.
3. **Check** the status and data with `expect`.

Only the database is faked. **Your real routes and controller run**, and that is what gets tested.

What we get as a result: a list of ✓ (pass) or ✕ (fail) in the terminal. A failing test shows what it expected and what it received.

Why this syntax:

Code	Why it is used
`describe("...", () => { })`	Groups related tests under a title
`test("...", () => { })`	One test: a readable name and code that checks one thing
`expect(a).toBe(b)`	Passes only if `a` equals `b`
`expect(list).toHaveLength(n)`	Passes if the list has `n` items
`request(app).get(url)`	Supertest sends a GET to the app (no server or port needed)
`.send(data)`	Adds a JSON body, for POST and PUT
`jest.mock(path, () => ({ ... }))`	Replaces the real Student model with a fake in this test file
`jest.fn()`	A fake function that remembers how it was called
`.mockResolvedValue(x)`	The fake returns `x`, as if MongoDB had answered
`.mockRejectedValue(err)`	The fake throws, to test the error path
`afterEach(() => jest.clearAllMocks())`	Reset the fakes after each test, so tests don't affect each other
`--verbose` (in `package.json`)	Shows each test's name, not just the totals

Example code

The simplest test: check a normal function. Use `studentUtils.js` from Node 1. Inside `practice`, run `npm install -D jest`.

`practice/studentUtils.test.js`

```
const { formatStudent, isCSE } = require("./studentUtils");

describe("studentUtils", () => {
  test("formats name and USN", () => {
    expect(formatStudent("Asha", "1RV23CS001")).toBe("Asha (1RV23CS001)");
  });

  test("knows which department is CSE", () => {
    expect(isCSE("CSE")).toBe(true);
    expect(isCSE("ECE")).toBe(false);
  });
});
```

Run it:

```
npx jest --verbose studentUtils.test.js
```

We name the file because the Mocha example in the next section also lives in `practice`. Without a name, Jest would try to run that Mocha file too and fail. In `backend`, the `testMatch` setting below solves this instead.

Result:

```
PASS ./studentUtils.test.js
  studentUtils
    ✓ formats name and USN
    ✓ knows which department is CSE

Tests: 2 passed, 2 total
```

Break it on purpose: change `toBe("Asha (1RV23CS001)")` to `toBe("Asha")` and run again. The test fails and shows `Expected: "Asha"` and `Received: "Asha (1RV23CS001)"`.

Task (StudentDesk)

Write API tests for your backend. You need `app.js` from MongoDB 2 and the `test` script from Setup 3.

- [ ] Add a `jest` section to `backend/package.json`, so Jest only looks in `tests/jest` (keep a comma after the `scripts` block):

```
"jest": {
  "testEnvironment": "node",
  "testMatch": ["**/tests/jest/**/*.test.js"]
},
```

- [ ] Create `backend/tests/jest/students.test.js` with the code below
- [ ] Run `npm test` inside `backend`. Expected: **4 passed**
- [ ] Break it: in `studentController.js` change `res.status(201)` to `res.status(200)`. Run `npm test`: the POST test fails with `Expected: 201, Received: 200`. Change it back
- [ ] Add one more test of your own: `PUT /api/students/abc` returns 404 when `Student.findByIdAndUpdate` resolves to `null`

`backend/tests/jest/students.test.js`

```
const request = require("supertest");

jest.mock("../../models/Student", () => ({
  find: jest.fn(), create: jest.fn(),
  findByIdAndUpdate: jest.fn(), findByIdAndDelete: jest.fn(),
}));

const Student = require("../../models/Student");
const app = require("../../app");

const asha = {
  name: "Asha",
  usn: "1RV23CS001",
  email: "asha@mail.com",
  department: "CSE",
};

afterEach(() => jest.clearAllMocks());

describe("GET /api/students", () => {
  test("returns all students", async () => {
    Student.find.mockResolvedValue([{ _id: "1", ...asha }]);

    const res = await request(app).get("/api/students");

    expect(res.status).toBe(200);
```

```
    expect(res.body).toHaveLength(1);
    expect(res.body[0].name).toBe("Asha");
  });
});

describe("POST /api/students", () => {
  test("saves a student and returns 201", async () => {
    Student.create.mockResolvedValue({ _id: "abc", ...asha });

    const res = await request(app).post("/api/students").send(asha);

    expect(res.status).toBe(201);
    expect(res.body._id).toBe("abc");
  });

  test("returns 400 when data is invalid", async () => {
    Student.create.mockRejectedValue(new Error("usn: Path `usn` is
required."));

    const res = await request(app).post("/api/students").send({ name: "Asha"
});

    expect(res.status).toBe(400);
  });
});

describe("DELETE /api/students/:id", () => {
  test("returns 404 when the student does not exist", async () => {
    Student.findByIdAndDelete.mockResolvedValue(null);

    const res = await request(app).delete("/api/students/abc");

    expect(res.status).toBe(404);
  });
});
```

Why `../../`: the test file is in `backend/tests/jest/`, so it goes up two folders to reach `backend/`, where `app.js` and `models/` are.

Testing 2: Mocha and Chai

Explanation

Why we do this: Mocha + Chai does the **same job** as Jest, using two packages instead of one. Many companies use it, so it's worth knowing both. Once you see them side by side, you'll notice the idea (fake → send → check) is identical; only a few words change.

Jest vs Mocha + Chai:

	Jest	Mocha + Chai
Packages	`jest`	`mocha` (runner) + `chai` (checker)
Group tests	`describe`	`describe`
One test	`test("...", ...)`	`it("...", ...)`
Check equal	`expect(x).toBe(5)`	`expect(x).to.equal(5)`
Check length	`toHaveLength(2)`	`to.have.lengthOf(2)`
Fake the database	`jest.mock(...)`	Swap the function by hand
Pass mark	✓	✔
Run (StudentDesk)	`npm test`	`npm run test:mocha`

Faking without `jest.mock`: Mocha has no built-in mocking, so we replace the model's function ourselves: save the real one in `beforeEach`, put a fake in the test, and restore the real one in `afterEach`. If we forgot to restore it, the fake would leak into the next test.

What we get as a result: a list of ✔ and "N passing". A failure shows a message like `expected 200 to equal 201`.

Why this syntax:

Code	Why it is used
`const { expect } = require("chai")`	Takes Chai's `expect` checker
`it("...", async () => { })`	One test (Mocha's word for `test`)
`expect(a).to.equal(b)`	Passes if `a` equals `b`. Chai reads like English

Code	Why it is used
`.to.include({ ... })`	Passes if the object contains those fields
`beforeEach(fn)`	Runs before every test: we save the real functions
`afterEach(fn)`	Runs after every test: we put them back
`Student.find = async () => [...]`	Our hand-made fake: it returns a list, like MongoDB would
`Object.assign(Student, original)`	Copies the saved real functions back onto `Student`
`mocha tests/mocha`	Runs every test file in that folder

Node version: recent Chai versions need **Node 22 or newer** to load with `require`. If you see `ERR_REQUIRE_ESM`, update Node to the current LTS (Setup 1).

Example code

Test `studentUtils.js` again, this time with Mocha + Chai. Inside `practice`, run `npm install -D mocha chai`.

`practice/studentUtils.spec.js`

```
const { expect } = require("chai");
const { formatStudent, isCSE } = require("./studentUtils");

describe("studentUtils (Mocha + Chai)", () => {
  it("formats name and USN", () => {
    expect(formatStudent("Asha", "1RV23CS001")).to.equal("Asha
(1RV23CS001)");
  });

  it("knows which department is CSE", () => {
    expect(isCSE("CSE")).to.equal(true);
    expect(isCSE("ECE")).to.equal(false);
  });
});
```

Run it:

```
npx mocha studentUtils.spec.js
```

Result:

```
  studentUtils (Mocha + Chai)
    ✔ formats name and USN
    ✔ knows which department is CSE

  2 passing
```

Compare it line by line with the Jest example: `test` became `it`, and `toBe` became `to.equal`.

Task (StudentDesk)

Write the same kind of API tests with Mocha + Chai.

- [ ] Create `backend/tests/mocha/students.spec.js` with the code below
- [ ] Run `npm run test:mocha` inside `backend`. Expected: **3 passing**
- [ ] Break the 201 again (as in Testing 1) and see Mocha's message: `expected 200 to equal 201`. Fix it
- [ ] Add your own test: `POST` with invalid data returns 400 (make `Student.create` throw an error)
- [ ] Commit to Git and push

`backend/tests/mocha/students.spec.js`

```
const request = require("supertest");
const { expect } = require("chai");
const Student = require("../../models/Student");
const app = require("../../app");

const asha = {
  name: "Asha",
  usn: "1RV23CS001",
  email: "asha@mail.com",
  department: "CSE",
};

describe("Student API (Mocha + Chai)", () => {
  const original = {};
```

```
  beforeEach(() => {
    original.find = Student.find;
    original.create = Student.create;
    original.findByIdAndDelete = Student.findByIdAndDelete;
  });

  afterEach(() => {
    Object.assign(Student, original);
  });

  it("GET returns all students", async () => {
    Student.find = async () => [{ _id: "1", ...asha }];

    const res = await request(app).get("/api/students");

    expect(res.status).to.equal(200);
    expect(res.body).to.have.lengthOf(1);
  });

  it("POST saves a student (201)", async () => {
    Student.create = async (data) => ({ _id: "abc", ...data });

    const res = await request(app).post("/api/students").send(asha);

    expect(res.status).to.equal(201);
    expect(res.body).to.include({ _id: "abc", usn: "1RV23CS001" });
  });

  it("DELETE unknown id gives 404", async () => {
    Student.findByIdAndDelete = async () => null;

    const res = await request(app).delete("/api/students/abc");

    expect(res.status).to.equal(404);
  });
});
```

Final task and common errors

Final task: Course Management API

You built the Student API step by step. Now build a **Course API** in the same `backend`, on your own, using the same pattern for every file.

Student file (done)	Course file (you build)
`models/Student.js`	`models/Course.js`
`controllers/studentController.js`	`controllers/courseController.js`
`routes/studentRoutes.js`	`routes/courseRoutes.js`
`app.use("/api/students", ...)` in `app.js`	`app.use("/api/courses", ...)` in `app.js`
`tests/jest/students.test.js`	`tests/jest/courses.test.js`

Requirements:

- [ ] Course schema: `title` (required), `code` (required, unique), `instructor` (required), `credits` (Number, `min: 1`, `max: 5`), `department` (enum `CSE`, `ISE`, `ECE`, `ME`, default `CSE`), with timestamps
- [ ] Routes: `GET /api/courses`, `GET /api/courses/:id`, `POST /api/courses`, `PUT /api/courses/:id`, `DELETE /api/courses/:id`
- [ ] Correct status codes: 200, 201, 400, 404
- [ ] Filter: `GET /api/courses?department=CSE` (hint: `Course.find(req.query.department ? { department: req.query.department } : {})`)
- [ ] At least 5 tests with Jest **or** Mocha + Chai, and `npm test` passes
- [ ] `.env` and `node_modules` are **not** on GitHub
- [ ] Push to GitHub and submit the repository link

Sample course for testing:

```
{
  "title": "Data Structures",
  "code": "CS301",
  "instructor": "Dr. Rao",
  "credits": 4,
  "department": "CSE"
}
```

Bonus: sort with `GET /api/courses?sort=credits` (hint: `Course.find().sort({ credits: 1 })`), or copy the React app and change it to show courses.

Common errors cheat sheet

Error or symptom	Cause	Fix
`Missing script: "dev"` or `"test"`	Script not in `package.json`, or not saved	Add it to `"scripts"`, save, check with `npm run`
`Could not read package.json`	Terminal is in the wrong folder	`cd backend` or `cd frontend` first
`Cannot find module '../models/Student'`	Wrong path or wrong capital letter	File must be `models/Student.js`; check `./` and `../`
`Permission denied` for `nodemon` or `vite`	Project is on an external/NTFS drive (Linux)	Move the project to your home folder and run `npm install` again
`EADDRINUSE: address already in use`	An old server is still running on that port	Ctrl + C in the old terminal, or change the port
`req.body` is `undefined`	`express.json()` missing or below the routes	`app.use(express.json())` above the routes
React: "NetworkError" / CORS error in Console	`cors()` missing, or backend not running	Add `app.use(cors())`; start the backend on port 5000
`Cannot DELETE /api/students`	No id in the URL	`DELETE /api/students/<_id>`

Error or symptom	Cause	Fix
400 with `SyntaxError: Unexpected token`	Invalid JSON in Thunder Client (often a missing comma)	Fix the JSON; no comma after the last field
`bad auth`	Wrong user or password in `MONGO_URI`	Check the Atlas database user; remove `< >`
Connection timeout to Atlas	Your IP is not allowed	Add `0.0.0.0/0` in Network Access
`E11000 duplicate key`	That USN already exists	Use a different USN
`ValidationError`	Required field missing or value not in `enum`	Send all fields with allowed values
Response is `{}`	`await` missing before a Mongoose call	Add `await`
`ERR_REQUIRE_ESM` when testing with Chai	Node is too old	Install the current Node LTS

Commands summary

Where	Command	What it does
`backend`	`npm run dev`	Start the API on port 5000 (restarts on save)
`frontend`	`npm run dev`	Start React on port 5173
`backend`	`npm test`	Run the Jest tests
`backend`	`npm run test:mocha`	Run the Mocha + Chai tests
`studentdesk`	`git add .` → `git commit -m "..."` → `git push`	Save and upload your work